

Multi-Armed Bandits

Sujit Prakash Gujar

sujit.gujar@iiit.ac.in

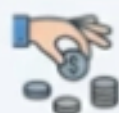
©Sujit Gujar



A word cloud representing Sujit Gujar's research interests. The words are arranged in a roughly circular shape, with 'Web-Applications' and 'Crowdsourcing' being the most prominent. Other visible words include 'Machine-Learning', 'Mechanism-Design', 'Google', 'Flipkart', 'MAB', 'Online-Auctions', 'Algorithms', 'Cryptogrphy', 'Private Information', 'Internet-Advertising', 'Matching-Markets', 'Healthcare', 'Amazon', 'Facebook', 'GameTheory', 'Linear Regression', 'Trustful', 'Online', 'UCB Healthcare', 'Strategy', 'Display Ads', and 'Algorithms'.

BANDIT PROBLEM CONCEPTS

POPULAR ALGORITHMS



Greedy



ϵ -Greedy



Upper-Confidence Bound (UCB)



Thomson Sampling

CLASSIFICATION



Single-pull

vs



Combinatorial



Always available

vs



Volatile



Stochastic

vs



Adversarial



Context free

vs



Context dependant Rewards



Stationary

vs



Non-Stationary

Image credit: Gemini AI

Motivation



Image Credits: Gemini, ChatGPT, Copilot

Multi-Armed Bandits



Introduction to MAB

Multi-Armed Bandit (MAB): a classic problem in decision-making under uncertainty

Key Elements:

- K arms (actions/choices)
- Each arm has a stochastic reward distribution, parameter unknown
- At each round, select one arm and observe reward
- **Goal:** Maximize cumulative reward over T rounds

The Core Challenge:

- Balance between exploration and exploitation
- Learn while optimizing

- It estimates the expected reward $\hat{\mu}_i$ for each arm
- At every t , it selects the arm with the **highest** $\hat{\mu}_i$.
- Focuses purely on **exploitation**

Greedy Strategy

Algorithm 1: Greedy Strategy [1]

- 1: **Input:** K arms
- 2: **Initialization:** Pull each arm $i \in [K]$ once to initialize $\hat{\mu}_i$.
- 3: **for** $t = K + 1, \dots, T$ **do**
- 4: Select arm $i_t \in \arg \max_{i \in [K]} \hat{\mu}_{i,t-1}$
- 5: Observe reward $r_{i_t,t}$
- 6: Update empirical mean $\hat{\mu}_{i_t}$ using $r_{i_t,t}$
- 7: **end for**

- The ϵ -Greedy algorithm gives a simple method for **exploration**
- At each step t , with probability $1 - \epsilon$, it chooses the greedy action
- With probability ϵ , it chooses an arm uniformly at random

Algorithm 2: ϵ -Greedy Strategy [1]

- 1: **Input:** K arms, exploration parameter $\epsilon \in (0, 1)$
- 2: **Initialization:** Pull each arm $i \in [K]$ once to initialize $\hat{\mu}_i$ and counts $\mathcal{N}_i$.
- 3: **for** $t = K + 1, \dots, T$ **do**
- 4: Sample $p \sim \text{Uniform}(0, 1)$
- 5: **if** $p < \epsilon$ **then**
- 6: Select arm $i_t \sim \text{Uniform}(\{1, \dots, K\})$ {Explore}
- 7: **else**
- 8: Select arm $i_t \in \arg \max_{j \in [K]} \hat{\mu}_{j,t-1}$ {Exploit}
- 9: **end if**
- 10: Observe reward $r_{i_t,t}$
- 11: Update count $\mathcal{N}_{i_t}(t) \leftarrow \mathcal{N}_{i_t}(t-1) + 1$
- 12: Update $\hat{\mu}_{i_t,t} \leftarrow \hat{\mu}_{i_t,t-1} + \frac{1}{\mathcal{N}_{i_t}(t)}(r_{i_t,t} - \hat{\mu}_{i_t,t-1})$
- 13: **end for**

Measuring Performance

Why do we need a performance measure?

- Need to quantify how well an algorithm performs
- Compare different algorithms objectively
- Understand the cost of learning

The Concept of Regret:

- Measures the **stochastic** difference between optimal and achieved performance
- Captures the “loss” due to not knowing the best arm initially
- Lower regret = better algorithm

Types of Regret

Definition (Expected Regret)

If $r_{i_t,t}$ denotes the reward observed by pulling arms according to an algorithm $\mathcal{A}$

$$Reg_{\mathcal{A}}^{\text{exp}}(T) = \mathbb{E} \max_{1, \dots, K} \left[\sum_{t=1}^T r_{i,t} - r_{i_t,t} \right]$$

Types of Regret

Definition (Pseudo Regret)

The pseudo regret of an algorithm $\mathcal{A}$ is,

$$Reg_{\mathcal{A}}^{\text{pseudo}}(T) = \sum_{t=1}^T \mathbb{E} \max_{1, \dots, K} r_i - \mathbb{E} \left[\sum_{t=1}^T r_{i_t, t} \right] = \sum_{t=1}^T \mathbb{E} [\mu_{j^*} - \mu_{A_t}] .$$

- $Reg_{\mathcal{A}}(T) = T \times \mu_{j^*} - \sum_{t=1}^T \mu_{i_t}$